# Notes de cours : Conteneurisation d'un modele de langage local (Ollama) et deploiement sur Render

## 1. Objectifs pedagogiques et mise en contexte

Ce cours s'adresse a des etudiants ayant une base en programmation Web avec JavaScript et Node.js. L'objectif est de demystifier le deploiement des modeles de langage (LLM) en montrant qu'il est possible d'empaqueter un moteur d'inference et une application cliente dans un conteneur standardisable, autonome et portable.

### Competences visees

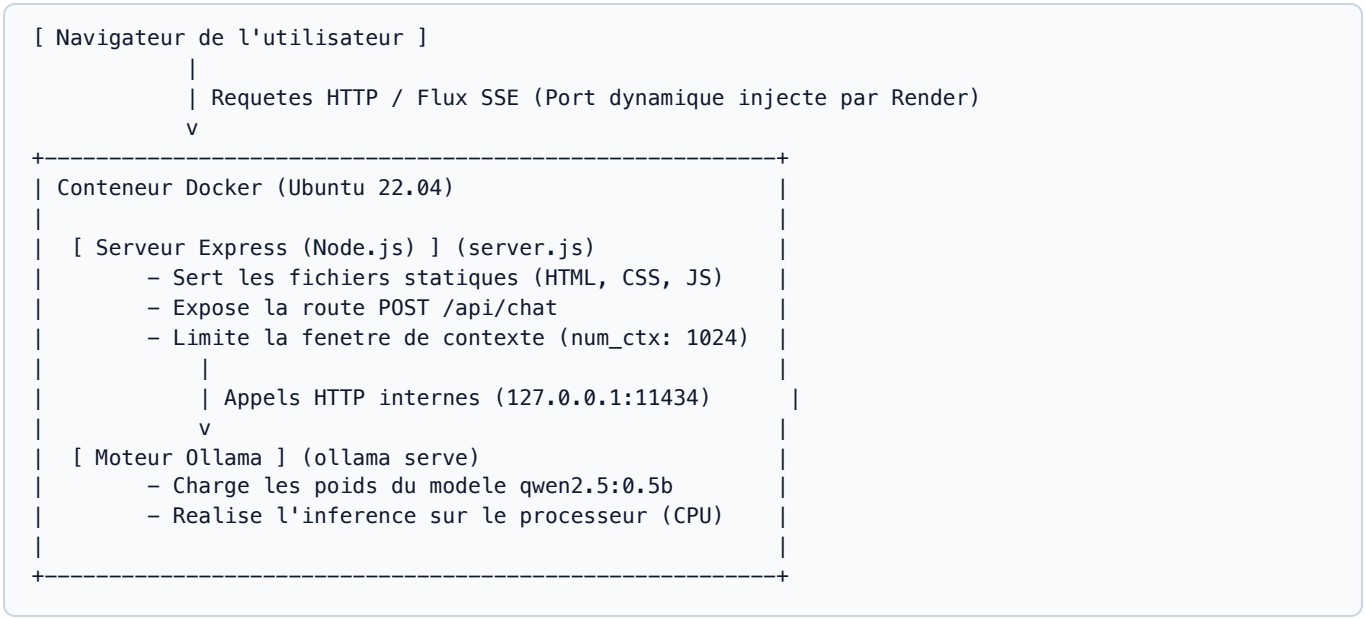
- Comprendre le fonctionnement d'un moteur d'inference local (Ollama) et son interface de programmation (API REST).
- Creer une interface Web de clavardage connectee a un flux de reponse en continu (Server-Sent Events).
- Concevoir un fichier Dockerfile qui combine deux services au sein d'un meme conteneur.
- Comprendre la difference entre le stockage persistant et l'inclusion prealable des donnees lourdes (le patron de conception "bake").
- Analyser et respecter les contraintes strictes d'un environnement infonuagique gratuit (memoire vive, processeur, mise en veille).
- Configurer et reussir le deploiement continu d'un conteneur vers la plateforme Render.

---

## 2. Architecture de la solution

Dans un environnement infonuagique mutualise comme Render (offre gratuite), chaque service Web correspond a un conteneur isole. Pour eviter d'avoir a payer plusieurs services distincts, nous adoptons une architecture ou le serveur d'application et le moteur d'inference cohabitent dans le conteneur.

### Schema des composants internes du conteneur




### Le role de chaque composant

- **Node.js / Express** : Agit comme serveur mandataire (proxy) et serveur Web. Il protege l'acces direct a Ollama, valide les requetes et sert l'interface graphique.
- **Ollama** : Fournit le moteur d'execution optimise en C++ (base sur llama.cpp). Il charge les poids du modele et effectue les calculs matriciels.
- **Le script d'entree (entrypoint.sh)** : Orchestre l'ordre de demarrage. Il lance d'abord Ollama en arriere-plan, verifie que le port 11434 repond, puis demarre Node.js au premier plan.

---

## 3. Les contraintes du nuage et le choix du modele Qwen 2.5 (0.5B)

Cette section constitue le coeur de l'apprentissage des realites industrielles du deploiement.

### A. La limite de memoire vive (512 Mo sur Render)

L'offre gratuite de Render alloue un maximum de **512 Mo de memoire vive (RAM)** par conteneur. Si le conteneur depasse cette limite, le systeme d'exploitation l'arrete immediatement avec le code d'erreur `137` (signal `SIGKILL` envoye par le gestionnaire OOM - Out Of Memory).

- Les modeles populaires comme Llama 3 (8 milliards de parametres) ou Mistral 7B necessitent entre 5 et 16 Go de memoire vive. Ils provoqueront un arret immediat.
- Le modele **Qwen 2.5 0.5B** ne compte qu'environ 500 millions de parametres. Ses poids quantifies au format Q4 occupent environ 398 Mo sur le disque.
- En fonctionnement, le partage de la memoire vive se repartit comme suit :
  - Systeme de base (Ubuntu/Linux) : environ 30 Mo
  - Runtime Node.js + Express : environ 35 Mo

- Processus Ollama au repos : environ 45 Mo
- Poids du modele en memoire + calculs d'inference : environ 330 a 380 Mo
- **Total estime** : environ 440 a 490 Mo. Le modele tient sous le plafond des 512 Mo, a condition de limiter le cache.

### B. L'optimisation cruciale du cache de contexte (`num_ctx`)

Par defaut, Ollama reserve une fenetre de contexte de 2048 ou 4096 jetons (tokens). Cette reserve alloue un tableau volumineux en memoire vive appele la memoire tampon KV (Key-Value cache).

- **Dans notre code (`server.js`)**, nous forcons le parametre :

```
options: {
  num_ctx: 1024,
  temperature: 0.7
}
```

- Cela permet de plafonner la taille de la memoire tampon et d'eviter le crash memoire lors de longues discussions.

### C. Le stockage ephemere et la technique du "Bake"

Sur Render (sans disque payant optionnel), le stockage d'un conteneur est ephemere. Si nous telechargions le modele au moment ou le conteneur demarre (`ollama pull` dans le script d'entree) :

1. Chaque redemarrage necessiterait le telechargement de 400 Mo.
2. Le temps de demarrage depasserait la limite permise par Render (timeout de demarrage).
3. La consommation de bande passante serait inutilement elevee.

**La solution : inclure le modele lors de la construction de l'image (le "Bake")** Dans le `Dockerfile`, nous lancions temporairement le moteur pendant l'etape de fabrication (`RUN ollama serve & ... && ollama pull qwen2.5:0.5b`). Le fichier de modele est ainsi grave de maniere permanente dans les couches de l'image Docker.

### D. Processeur partage et vitesse d'inference

Render n'offre pas de processeur graphique (GPU) sur son plan gratuit. Tous les calculs se font sur un processeur virtuel partage (vCPU).

- Avec un modele de 0.5 milliard de parametres, la vitesse d'inference reste acceptable (de 5 a 12 jetons par seconde).
- L'utilisation du protocole Server-Sent Events (SSE) permet d'afficher les mots au fur et a mesure de leur generation, ce qui donne une impression de fluidite a l'utilisateur malgre l'absence de GPU.

### E. La mise en veille (Cold Start)

Apres 15 minutes d'inactivite, Render eteint le conteneur. Lors de la prochaine visite, Render relance l'image. Le premier chargement prend entre 45 et 90 secondes. Il est important d'en avertir les etudiants pour qu'ils

ne croient pas que leur deploiement a echoue.

---

## 4. Analyse detaillee des fichiers cles

### Le fichier `Dockerfile`

1. **Image de depart ( `FROM ubuntu:22.04` )** : Offre une compatibilite sans faille avec le script d'installation binaire officiel d'Ollama.
2. **Dependances systeme ( `curl` , `ca-certificates` , `zstd` ) et Node.js 20** : Le paquet `zstd` est requis par Ollama pour decompresser ses archives binaires au format Zstandard. Node.js 20 LTS fournit le moteur `fetch` natif.
3. **Installation d'Ollama** : `curl -fsSL https://ollama.com/install.sh | sh`.
4. **Pre-telechargement du modele** : Un processus d'arriere-plan est demarre, attend que le port local 11434 soit ouvert, telecharge le modele dans `/root/.ollama/models`, puis s'arrete proprement.
5. **Separation des etapes npm** : On copie d'abord `package.json` et on execute `npm install` avant de copier le reste du code. Cela exploite le systeme de cache de couches de Docker (si l'etudiant modifie le HTML ou le CSS, Docker ne reinstalle pas les modules npm).

### Le script d'entree `entrypoint.sh`

1. `set -e` : Interrompt le script immediatement si une commande echoue.
2. `ollama serve &` : Lance le service d'inference en arriere-plan.
3. Boucle d'attente active : Interroge `http://127.0.0.1:11434/api/tags` a l'aide de curl jusqu'a ce qu'Ollama confirme son bon fonctionnement.
4. `PORT_APPLICATION=${PORT:-3000}` : Recupere la variable `$PORT` injectee dynamiquement par Render.
5. `exec node server.js` : L'instruction `exec` remplace le processus shell par le processus Node.js. C'est essentiel pour que Node.js recoive directement les signaux d'arret ( `SIGTERM` ) emis par Docker ou Render lors des deploiements.

### Le serveur Express `server.js`

- L'ecoute se fait obligatoirement sur `0.0.0.0` et non sur `localhost` ou `127.0.0.1`. Dans un conteneur, ecouter sur `127.0.0.1` empeche tout trafic externe d'atteindre l'application.
- La route `/api/status` permet d'indiquer dans l'interface si Ollama est pret a repondre.
- Le flux `text/event-stream` transmet chaque paquet genere par Ollama au navigateur sans attendre la fin complete de la phrase.

---

## 5. Deroulement suggere d'une seance de cours (3 heures)

### Periode 1 (45 minutes) : Concepts et decouverte

- Presentation du concept d'inference locale face aux interfaces de programmation proprietaires (OpenAI, Anthropic).
- Demonstration d'Ollama en ligne de commande.
- Explication de la quantification (Q4) et de la relation entre le nombre de parametres et l'empreinte memoire vive.
- Revue du code JavaScript (Express, requete vers Ollama, flux de lecture).

### Periode 2 (45 minutes) : La conteneurisation avec Docker

- Revue ligne par ligne du `Dockerfile` et du script `entrypoint.sh`.
- Construction de l'image en local : `docker build -t mini-chatbot .`
- Explication des couches de construction et de la mise en cache.
- Lancement et test en local : `docker run -p 3000:3000 mini-chatbot`

### Periode 3 (60 minutes) : Deploiement sur Render

- Creation d'un depot Git et publication sur GitHub.
- Connexion du compte GitHub a Render.
- Creation du "Web Service" base sur Docker.
- Suivi des journaux d'evenements (build logs et runtime logs).
- Test de l'application en direct sur l'adresse URL publique `onrender.com`.

### Periode 4 (30 minutes) : Debriefing et experimentation

- Observation de la consommation memoire dans le tableau de bord de Render.
- Test de provocation de depassement de memoire (que se passe-t-il si on tente de charger un modele de 7B ?).
- Discussion sur les alternatives industrielles (serveurs dedies avec GPU, services de conteneurs sans serveur comme Fly.io ou AWS ECS).

---

## 6. Guide de depannage pour les etudiants

### 1. Erreur 137 (OOM - Out of Memory)

- **Symptome** : Le conteneur s'eteint subitement au moment ou l'etudiant pose une question. Dans les journaux de Render, le message `Exited with status 137` apparait.
- **Cause** : La memoire vive a depasse 512 Mo.

- **Verification** : S'assurer que le modele utilise est bien `qwen2.5:0.5b` et que la variable `num_ctx` est bien reglee a 1024 dans `server.js`.

### 2. Le deploiement echoue pendant le build (Timeout de Render)

- **Symptome** : Render annule la construction apres 15 minutes.
- **Cause** : Le telechargement du modele pendant la construction a ete ralenti par le reseau ou l'image est trop lourde.
- **Solution** : Relancer un deploiement manuel depuis le tableau de bord Render ("Clear build cache & deploy").

### 3. L'application est inaccessible ("Bad Gateway" ou page blanche)

- **Symptome** : L'URL de Render renvoie une erreur 502.
- **Cause** : Le serveur Node.js n'ecoute pas sur le bon port.
- **Solution** : Verifier que `server.js` lit bien `process.env.PORT` et qu'il ecoute sur `0.0.0.0`.

### 4. Particularites et pieges sous Windows

- **Fins de ligne (CRLF vs LF)** : Si un etudiant modifie `entrypoint.sh` sous Windows avec un editeur qui insere des retours `CRLF` (`\r\n`), le conteneur Linux refusera de demarrer avec l'erreur `/bin/bash^M: bad interpreter`. Le `Dockerfile` integre une commande `sed` pour nettoyer automatiquement ces caracteres, et le fichier `.gitattributes` force les fins de ligne Unix.
- **Moteur Docker Desktop** : L'etudiant sous Windows doit utiliser le backend WSL 2 dans les options de Docker Desktop.
- **Invite de commandes** : Recommander **PowerShell** ou **Windows Terminal**. Les commandes `docker build` et `docker run` sont rigoureusement identiques a macOS ou Linux.